

# Anwendung, Grenzen und Zukunft

## Die Azubi-Methode, Teil 3

*Allgemeine Anwendbarkeit, LLM-Integration und offene Fragen*

### Allgemeine Anwendbarkeit

Die Azubi-Methode ist nicht auf eine einzelne Domäne beschränkt. Sie lässt sich auf jedes Ergebnis anwenden, das von einem Experten oder einem Experten-System produziert wurde. Der Grund ist einfach: Stille Annahmen sind kein domänenspezifisches Problem. Sie sind ein Wissens-Problem. Überall dort, wo Expertise eingesetzt wird, entstehen blinde Flecken. Und überall dort, wo blinde Flecken entstehen, können kontextfreie Fragen sie sichtbar machen.

**Bewertungen und Analysen.** Ein AI-Agent erstellt eine Risikoeinschätzung. Der Azubi fragt: „Drei Risiken wurden als ›gering‹ eingestuft — nach welchem Maßstab?“ Die Frage erzwingt Transparenz über die Bewertungskriterien. Oft stellt sich heraus, dass der Maßstab nirgends definiert ist — er war eine stille Annahme des Experten.

**Prüfberichte und Audits.** Ein Prüfsystem bewertet ein Ergebnis gegen einen Katalog. Der Azubi fragt: „14 Prüfpunkte wurden übersprungen — steht das im Bericht?“ Die Frage deckt auf, was der Bericht nicht sagt. In der Praxis ist das, was ein Audit nicht geprüft hat, oft wichtiger als das, was er geprüft hat.

**Sicherheitsanalysen.** Ein Sicherheitsagent prüft ein System und meldet keine Auffälligkeiten. Der Azubi fragt: „Warum wurde Bereich X nicht geprüft?“ Die Frage macht sichtbar, was außerhalb des definierten Prüfbereichs lag. Die Grenze des Prüfbereichs ist selbst eine Annahme, die selten hinterfragt wird.

**Erkennung bössartiger Muster.** Ein LLM analysiert Daten auf Anomalien und findet nichts Verdächtiges. Der Azubi fragt: „Warum wird dieses Muster als normal behandelt? Was wäre, wenn es das nicht ist?“ Die Frage stellt die Definition von „normal“ in Frage — und genau dort können sich bössartige Muster verbergen, die deshalb unentdeckt bleiben, weil sie innerhalb der erwarteten Parameter liegen.

**Strategische Entscheidungen.** Ein AI-Agent erstellt eine Empfehlung. Der Azubi fragt: „Die Empfehlung basiert auf drei Annahmen — was passiert, wenn die zweite nicht stimmt?“ Die Frage macht stille Abhängigkeiten sichtbar.

**Zusammenfassungen und Konsolidierungen.** Ein Agent fasst Ergebnisse aus mehreren Quellen zusammen. Der Azubi fragt: „Quelle A sagt X und Quelle B sagt Y

— warum steht in der Zusammenfassung nur X?“ Die Frage deckt auf, ob der konsolidierende Agent einen Widerspruch geglättet hat, statt ihn sichtbar zu machen.

**Der Rahmen selbst.** Wer ein Regelwerk erstellt hat, kann den Azubi fragen, ob das Regelwerk Sinn ergibt. Die Methode kann sich selbst hinterfragen, und genau das macht sie robust. Ein System, das auch seine eigenen Grundlagen dem naiven Blick aussetzt, hat weniger blinde Flecken als eines, das nur seine Ergebnisse prüft.

Das Muster ist immer dasselbe: Die Aufgabenstellung und das Ergebnis sind die einzigen Eingaben. Der Azubi weiß nichts über den Weg dazwischen. Und genau das befähigt ihn, die Fragen zu stellen, die jeder, der den Weg kennt, nicht mehr stellt.

## Verwandte Ansätze

Die Azubi-Methode ist verwandt mit mehreren bestehenden Ansätzen, unterscheidet sich aber in einem spezifischen Punkt:

Die Sokratische Methode stellt gezielte, informierte Fragen, um den Gesprächspartner zum Nachdenken zu bringen. Die Azubi-Methode stellt naive Fragen ohne Kontext. Der Fragende weiß nicht, was die „richtige“ Antwort ist. Rubber-Duck-Debugging erzwingt Selbstreflexion durch Erklären. Die Azubi-Methode stellt Fragen von außen, ohne dass der Prüfer seinen eigenen Prozess erklären muss. Der Fresh-Eyes-Review — ein neuer Reviewer schaut auf das Ergebnis — ist das menschliche Äquivalent. Die Azubi-Methode formalisiert und systematisiert dieses Prinzip und macht es mit AI-Agenten wiederholbar und verschleißfrei. Socratic Debugging (Al-Hossami et al., 2023) nutzt sokratisches Fragen, um Programmieranfänger zum Finden eigener Bugs zu führen. Es zielt auf Lernen, nicht auf Qualitätssicherung.

Nach meiner Recherche gibt es keinen formalisierten Ansatz, der strukturierte Naivität ohne Kontext als systematischen Prüfmechanismus einsetzt. Die Azubi-Methode ist in diesem spezifischen Punkt — kontextfreies Hinterfragen als architektonisches Prinzip — ein eigenständiger Beitrag.

## Grenzen — ehrlich benannt

Die Azubi-Methode findet stille Annahmen. Sie findet keine Probleme, die nur unter realen Bedingungen sichtbar werden. Keine Usability-Probleme mit echten Nutzern. Keine Schwachstellen, die nur durch aktives Testen unter Belastung erscheinen.

Die beobachtete 100%-Trefferquote — keine einzige Azubi-Frage war überflüssig — basiert auf über 30 Fragen in einem spezifischen Kontext. Ob dieselbe Quote in anderen Domänen erreicht wird, ist eine offene Frage, die ehrlich als unbeantwortet markiert bleibt. Die Methode findet, was sie findet. Sie garantiert nicht, dass es nichts gibt, was sie nicht findet. Kein Werkzeug kann das.

Die Methode ist ein Werkzeug, nicht eine Lösung. Sie deckt eine spezifische Fehlerklasse ab, die andere Methoden nicht abdecken. Sie macht andere Methoden nicht überflüssig. Und sie macht den Menschen nicht überflüssig — am Ende entscheidet der Mensch, was er mit den Fragen anfängt.

Und: Alles hängt vom konkreten Sprachmodell ab. Wie gut ein Azubi-Agent fragt, wie tief ein Experten-Agent prüft, wie anfällig ein Modell für Sycophancy oder Halluzinationen ist — das variiert zwischen Modellen und sogar zwischen Versionen desselben Modells. Die Methoden in diesem Paper adressieren Probleme, die Stand heute real und messbar sind. Ob sie in zwei Jahren noch im selben Maß gelten, ist eine offene Frage.

## Für LLM-Hersteller

Die Azubi-Methode beschreibt einen Mechanismus, der heute manuell angewendet werden muss. Der Entwickler baut sich seine kontextfreie Instanz, öffnet einen neuen Chat, kopiert Aufgabenstellung und Ergebnis, bittet um Fragen. Das funktioniert. Aber es skaliert nicht. Hundert Teams, die unabhängig voneinander denselben Mechanismus nachbauen, sind hundert Einzellösungen, die alle unterschiedlich gut funktionieren.

Die Azubi-Methode ist der naheliegendste Kandidat für eine direkte Integration in Sprachmodelle: Ein Modell, das nach jedem Experten-Output automatisch einen kontextfreien Selbst-Check durchführt. Eine interne Instanz, die nur das Ergebnis sieht und eine Gegenfrage stellt. Das würde den manuellen Schritt überflüssig machen und die Methode jedem Nutzer zugänglich machen, ohne dass er sie kennen muss.

Die Vision wäre: Ein Modell, das nach einem Review automatisch eine kontextfreie Gegenfrage stellt, bräuchte keinen externen Azubi mehr. Ein Modell, das bei wiederholten Prüfungen bewusst seine Perspektive wechselt, hätte Rotation als

Default. Wenn das passiert, wäre dieses Paper nicht mehr eine Anleitung, sondern eine Beschreibung dessen, was Modelle ohnehin tun. Das wäre das bestmögliche Ergebnis.

Darüber hinaus gibt es drei konkrete Bereiche, an denen Hersteller arbeiten können:

**Kontextverständnis über lange Eingaben.** Modelle werden mit wachsendem Kontext nicht besser, sondern schlechter — ein Phänomen, das in der Forschung als Context Rot beschrieben wird. Besseres Kontextverständnis über die gesamte Fensterlänge würde die Wirksamkeit jeder Prüfmethode erhöhen.

**Resistenz gegen Context Poisoning.** Wenn eine falsche Annahme in den Kontext gelangt und sich über nachfolgende Antworten verstärkt, vergiftet sie das gesamte System. Modelle, die erkennen, wenn ein Kontexteintrag im Widerspruch zu ihrem eigenen Wissen steht, und diesen Widerspruch benennen statt bestätigen, wären ein fundamentaler Fortschritt.

**Reduzierte Sycophancy in Multi-Agent-Szenarien.** Die Bestätigungstendenz zwischen Agenten ist ein potenziell größeres Problem als die zwischen Mensch und AI. Modelle zu entwickeln, die in Agent-zu-Agent-Kommunikation kritischer statt zustimmender werden, wäre ein Hebel, der weit über dieses Paper hinausreicht.

## Die Philosophie dahinter

Die Grundlage ist 2.500 Jahre alt. Sokrates behauptete, nichts zu wissen. Und stellte deshalb die Fragen, die seine Gesprächspartner zwangen, ihre eigenen Annahmen zu überprüfen.

Die Azubi-Methode ist die Systematisierung dieses Prinzips für das Zeitalter von AI-Agenten: Wissen erzeugt blinde Flecken. Unwissenheit erzeugt Fragen. Und Fragen finden mehr als Antworten.

## Fazit

Jede Prüfmethode macht das Raster enger, nicht lückenlos. Automatisierte Prüfungen finden bekannte Muster. Experten finden erwartbare Logikfehler. Die Azubi-Methode findet stille Annahmen — die Fehler, die keine Fehlermeldung erzeugen, kein Warning, kein sichtbares Symptom. Die Fehler, die wochen- oder monatelang unbemerkt im System leben.

Die Methode ist heute anwendbar. Ohne Infrastruktur, ohne Budget, ohne Spezialwissen. Ein neuer Chat, eine Aufgabenstellung, ein Ergebnis und die Bitte um

Fragen. Zwei Minuten. Ein API-Call. Und die ehrliche Bereitschaft, sich mit den Antworten auseinanderzusetzen.

Es gibt keine 100%. Es gibt nur engere Maschen. Und jede Masche, die einen Fehler fängt, der vorher durchgerutscht wäre, hat sich gelohnt.

*Die richtigen Fragen sind wertvoller als die richtigen Antworten.*

# Quellen

## Grundlegende Konzepte

ELIZA-Effekt: Weizenbaum, J. (1966). „ELIZA — A Computer Program for the Study of Natural Language Communication Between Man and Machine.“ Communications of the ACM, 9(1), 36–45.

Groupthink: Janis, I. L. (1972). „Victims of Groupthink: A Psychological Study of Foreign-Policy Decisions and Fiascos.“ Houghton Mifflin.

Sokratische Methode: Platon, ca. 399 v. Chr. Überliefert in den Dialogen, insbesondere der Apologie des Sokrates.

## AI-Code-Qualität

CodeRabbit (2025). „State of AI vs Human Code Generation Report.“ 470 reale Pull Requests.

Faros AI (2025). „The AI Productivity Paradox Research Report.“ 10.000+ Entwickler.

METR (2025). Randomisierte Studie mit erfahrenen Open-Source-Entwicklern. 19% langsamer, subjektiv 20% schneller empfunden.

## Sycophancy und Reward Hacking

Sharma, M. et al. (2023). „Towards Understanding Sycophancy in Language Models.“ ICLR 2024.

Lechmazur Sycophancy Benchmark (2026). Contrarian Contradiction.

Beigi, M. et al. (2025). „SMART: Sycophancy Mitigation Through Reinforcement Learning.“

Denison, C. et al. (2024, Anthropic). „Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models.“

## Context-Probleme in LLMs

Chroma Research (2025). „Context Rot: How Increasing Input Tokens Impacts LLM Performance.“

Breunig, D. (2025). „How Long Contexts Fail.“

## **Verwandte Ansätze**

Al-Hossami, E. et al. (2023). „Socratic Questioning of Novice Debuggers.“ BEA Workshop, ACL 2023.

## Über den Autor

Andreas Schmidt forscht privat an AI-Anwendungen und ist nebenberuflich in der Software-Entwicklung tätig. Mit Andreas Schmidt Arts vereint er Marketing, Design und Coding. Die in diesem Paper beschriebene Methode entstand aus praktischer Erfahrung mit AI-Agenten-Systemen in der Software-Qualitätssicherung.

*Kontakt: [info@andreas-schmidt-arts.de](mailto:info@andreas-schmidt-arts.de)*

## Transparenzhinweis

Bei der Erstellung dieses Papers hat ein Sprachmodell unterstützt. Die Konzepte, Beobachtungen und Methoden stammen aus der praktischen Arbeit des Autors. Die Strukturierung, Formulierung und Recherche der Quellen wurde in Zusammenarbeit mit einem LLM durchgeführt. Alle Inhalte wurden vom Autor geprüft, gesteuert und verantwortet.